ICS: A Formal Instruction Contract Specification for Large Language Model Interfaces

Version 0.1 — Technical Report

Rahul Jaiswal[1]

[1]Vymo Inc.

February 26, 2026

Abstract

Instructions to Large Language Models (LLMs) are typically authored as informal prose, resulting in recurring failure modes: implicit constraints that disappear across sessions, mixed-lifetime context that cannot be cached, ambiguous output expectations that are resolved only by repeated re-prompting, and prompt content that cannot be audited or validated programmatically. We introduce the *Instruction Contract Specification* (ICS), a five-layer structured format that treats LLM instructions as formal interfaces rather than natural language requests. ICS separates instruction content by *lifetime*: permanent context, capability declarations, session state, per-invocation tasks, and output contracts. This separation enables prompt-caching APIs to serve stable content at reduced rates. We provide a formal grammar for capability directives, a conformance checker, and empirical evaluation on a real-world B2B payments platform workload across two providers. ICS-structured instructions achieve a **77.8%** reduction in effective API cost on Anthropic `claude-haiku-4-5-20251001` and a **15.3%** reduction on OpenAI `gpt-4o-mini` compared to the naïve flat-prompt baseline, with a proven lower bound of 55% token savings at N=10 that is independent of model or counting method. The provider difference is explained by cache rate (0.10× vs. 0.50×) and whether the naïve approach benefits from automatic caching. ICS is model-agnostic, framework-independent, and available as an open specification.

1 Introduction

The software industry has spent decades developing discipline around *interfaces*: API contracts, schemas, protocols, and type systems that make the behavior of software components explicit, auditable, and maintainable. These tools exist because interfaces that rely on shared context, informal conventions, and implied constraints do not compose and do not scale.

LLM instructions have not received the same treatment. The in-context learning paradigm established by Brown et al. [2020] showed that instructions and examples, provided as plain text in the model's context, can direct a wide range of behaviours without any weight update. This discovery was enormously powerful, but it also established an informal default: instructions as prose. In production systems, instructions are typically written as natural language prose, structured loosely by convention, and stored in files with no machine-readable semantics. When a system that uses an LLM instruction misbehaves, diagnosing whether the fault lies in the model, the instruction, or the caller's interpretation of the output requires manual inspection. When the instruction must be reused in a new context or maintained by someone who was not involved in writing it, implicit

assumptions collapse silently.

This produces four failure modes that recur across real deployments:

1. **Context collapse.** Long-lived facts (system invariants, architectural constraints) are mixed with short-lived context (session decisions, per-task parameters) into a single prompt block. Callers cannot determine what can be safely cached and what must be re-sent on every invocation, resulting in either unnecessary token cost or stale-context bugs.

2. **Implicit constraints.** Team conventions—"never modify the API layer," "always return amounts in minor units," "assume Python 3.11"—are documented in READMEs or exist only in the knowledge of the original author. When the instruction is reused or transferred, these constraints disappear silently.

3. **Ambiguous output expectations.** Instructions describe *what to do* without specifying what a correct result looks like. Output evaluation becomes subjective, failure states are undetectable by callers, and retry behavior is undefined.

4. **Retry as a first resort.** Ambiguous instructions produce inconsistent outputs. The standard response is to rephrase and retry—which treats ambiguity as a model failure rather than an interface failure, and produces no durable fix.

We argue that these failure modes share a root cause: *LLM instructions are not treated as interfaces.* The solution is not better prompting intuition—it is applying the same engineering discipline to instructions that is already applied to APIs, schemas, and protocols.

Contributions. We present the *Instruction Contract Specification* (ICS), which makes the following contributions:

1. A five-layer instruction format that separates content by lifetime (§3), making stable context structurally distinct from per-invocation content.

2. A formal directive grammar for capability declarations (§3.2.1), replacing prose constraints with machine-verifiable directives.

3. A required output contract with enumerated fields, eliminating ambiguous output expectations and undefined failure states (§3.5).

4. Empirical evidence that ICS's lifetime separation enables prompt-caching APIs to reduce effective API cost by 77.8% on a real-world workload, with a proven token-saving lower bound that holds for any conformant instruction (§5).

5. An open specification, reference validator, and worked examples across five real-world domains (§4).

2 Related Work

Prompt templates and structured prompting. Early systematic approaches to prompting introduced reusable templates and labeling schemes [Bach et al., 2022, Sanh et al., 2022]. DSPy [Khattab et al., 2023] compiles natural-language signatures into optimized prompt programs, treating prompt construction as a differentiable pipeline. LMQL [Beurer-Kellner et al., 2023] and Guidance [Guidance-AI contributors, 2023] add programming-language constructs (loops, conditionals, type constraints) to prompt authoring. TypeChat [Sweeney et al., 2023] uses TypeScript type

definitions to constrain LLM outputs to well-typed JSON. ICS differs from all of these: it does not optimize prompts, does not compile them, and does not constrain *model decoding*. Instead, ICS defines a *structural contract* for the *instruction itself*—what the caller commits to providing and how it is organized—independent of the underlying model or inference engine.

Chain-of-thought and instruction tuning. Chain-of-thought prompting [Wei et al., 2022b] and related techniques [Kojima et al., 2022, Wang et al., 2022] elicit multi-step reasoning by structuring the prompt's *content*. Instruction tuning [Ouyang et al., 2022, Wei et al., 2022a] aligns model behavior with natural-language instructions at training time. These methods address how models *reason*; ICS addresses how instructions are *structured and validated*. The two concerns are orthogonal: an ICS-compliant instruction can include chain-of-thought examples in its TASK_PAYLOAD layer.

Memory and context management. MemGPT [Packer et al., 2023] introduces a hierarchical memory system that pages context in and out of an LLM's context window at runtime, enabling unbounded effective context. Retrieval-augmented generation (RAG) [Lewis et al., 2020] injects retrieved documents into the prompt at query time. Both address *what content* the model sees; ICS addresses *how that content is organized and classified by lifetime*. The two approaches are complementary: a RAG system could use ICS to separate retrieved content (TASK_PAYLOAD) from stable system context (IMMUTABLE_CONTEXT).

Prompt caching and token efficiency. Anthropic's prompt caching API [Anthropic, 2024] and analogous mechanisms in other providers allow prefill KV-cache reuse for stable prompt prefixes. Gim et al. [Gim et al., 2023] propose prompt cache architectures for shared prefix reuse across requests. ICS's lifetime-separation design directly enables these mechanisms by making the stable prefix structurally explicit—the IMMUTABLE_CONTEXT and CAPABILITY_DECLARATION layers are always positioned first and never change across invocations in a session, making them natural cache targets. OpenAI's automatic prefix caching [OpenAI, 2024] caches prefixes of $\geq$ 1,024 tokens with no explicit markup required; ICS's non-redefinition rule is precisely the structural guarantee that keeps prefixes stable across invocations, preventing accidental cache invalidation.

Formal specifications for LLMs. Closest to ICS in spirit is TypeChat [Sweeney et al., 2023], which uses type schemas to constrain outputs, and work on schema-constrained decoding [Willard and Louf, 2023, Geng et al., 2023]. These focus on *output* structure; ICS defines contracts for both input and output. There is, to our knowledge, no prior work that defines a formal *multi-layer input specification* for LLM instructions with explicit lifetime semantics, directive syntax, and a conformance model.

Prompt sensitivity and reliability. Sclar et al. [2024] show that open-source LLMs shift task accuracy by up to 76 percentage points in response to trivial prompt formatting changes (separators, spacing, capitalisation), with no change in semantic content. Brown et al. [2020] established that unstructured in-context examples encode task behavior, implicitly confirming that prompt *form* matters as much as prompt *content*. ICS responds to this brittleness structurally: mandatory canonical ordering, explicit constraints, and non-redefinition rules eliminate the degrees of freedom that these sensitivity studies show are dangerous.

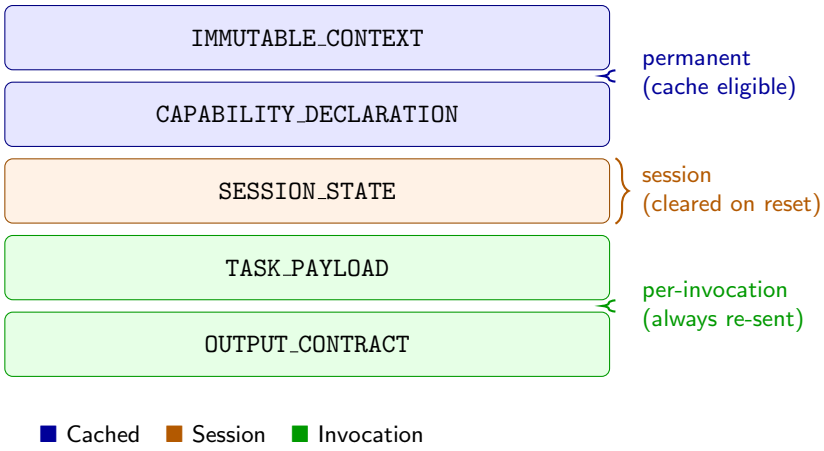

Figure 1: The five ICS layers in canonical order. Blue layers form a stable prefix eligible for KV-cache reuse. The orange layer accumulates session decisions and is cleared on session reset. Green layers change on every invocation and are always re-sent at full cost.

Context position and attention. Liu et al. [2024a] demonstrate a U-shaped utilisation curve in long-context LLMs: models attend most reliably to content at the very beginning and very end of the context window, degrading substantially for material placed in the middle. ICS's layer ordering—IMMUTABLE_CONTEXT first, OUTPUT_CONTRACT last, TASK_PAYLOAD in the penultimate position—aligns critical content with the positions models attend to most reliably. The non-redefinition rule prevents constraints from being accidentally buried in the middle by restating them near the task.

Prompt injection. Liu et al. [2024b] provide the first formal definition and systematic benchmark of prompt injection attacks. ICS's CAPABILITY_DECLARATION layer creates a structural permission boundary: injected content in TASK_PAYLOAD cannot override `DENY` directives declared in an earlier layer, because the non-redefinition rule (ICS §4.2) prohibits later layers from contradicting earlier constraints. ICS thus provides a specification-level defence against the mechanism by which injection succeeds in flat-prompt architectures.

3 The ICS Specification

An ICS-compliant instruction is composed of exactly five layers, appearing in the canonical order defined below. Each layer is delimited by boundary markers:

```
###ICS:<LAYER_NAME>###
<layer content>
###END:<LAYER_NAME>###
```

Layers MUST appear in order. A parser encountering layers out of order MUST reject the instruction as non-conformant. Figure 1 illustrates the five layers, their lifetimes, and which layers are eligible for KV-cache reuse across invocations. The five layers and their lifetime semantics are as follows.

3.1 IMMUTABLE_CONTEXT

IMMUTABLE_CONTEXT declares long-lived facts that remain constant across all tasks within a domain: system description, repository or schema layout, architectural invariants, external dependency contracts. It MUST NOT include task-specific instructions. It is the only layer eligible for aggressive prompt caching, because it is stable across sessions by definition.

Key rules. Content MUST NOT be restated, redefined, or contradicted in any subsequent layer. Any content that would be valid in IMMUTABLE_CONTEXT but is placed in SESSION_STATE or TASK_PAYLOAD is a conformance violation.

3.2 CAPABILITY_DECLARATION

CAPABILITY_DECLARATION defines what the model is permitted, forbidden, and required to do. All permissions are expressed using three directives: `ALLOW`, `DENY`, and `REQUIRE`. Undeclared actions are DENIED by default.

3.2.1 Directive grammar

```
directive  ::= KEYWORD WS+ action (WS+ qualifier)? (WS+ "IF" WS+ condition
   )?
KEYWORD    ::= "ALLOW" | "DENY" | "REQUIRE"
action     ::= WORD (WS+ WORD)*
qualifier  ::= QWORD WS+ target
QWORD      ::= "WITHIN" | "ON" | "WITH" | "UNLESS"
target     ::= WORD (WS+ WORD)*
condition  ::= WORD (WS+ WORD)*
WORD       ::= [A-Za-z0-9_./-]+
WS         ::= " " | "\t"
```

The grammar is intentionally liberal on scope expressions to support existing codebase idioms without requiring rewrites at adoption time. A normative scope grammar is deferred to ICS v0.2 as an optional conformance level. Example directives:

```
ALLOW   file modification WITHIN src/orders/
DENY    modification of src/orders/api/
DENY    introduction of float arithmetic ON monetary values
REQUIRE type annotations ON all new functions
REQUIRE backward compatibility WITH gateway/v2/ API contracts
```

3.3 SESSION_STATE

SESSION_STATE captures temporary assumptions and decisions that accumulate within a session but must not outlast it: conclusions reached in prior invocations, temporary flags, confirmed facts about the current task scope. Each entry SHOULD carry an identifier or timestamp.

A session ends when SESSION_STATE contains only the sentinel value `CLEAR`:

```
###ICS:SESSION_STATE###
CLEAR
###END:SESSION_STATE###
```

A SESSION_STATE layer containing `CLEAR` alongside other content MUST be rejected as malformed. Declaring a new IMMUTABLE_CONTEXT implicitly clears any prior SESSION_STATE.

3.4 TASK_PAYLOAD

TASK_PAYLOAD is the only layer that changes per invocation. It specifies the precise action to be performed, without restating context from preceding layers. A TASK_PAYLOAD is executable without clarification if all required inputs, constraints, and success conditions are fully defined across the instruction layers.

3.5 OUTPUT_CONTRACT

OUTPUT_CONTRACT declares the required shape of the response. It is response- scoped but MUST be declared before execution. All four fields are required:

Field	Meaning
`format`	Output encoding (JSON, unified diff, prose, ...)
`schema`	Structure definition (JSON Schema, field list, ...)
`variance`	Explicitly permitted deviations; unlisted variance is forbidden
`on_failure`	Required behavior when output cannot conform

Free-form prose is invalid unless `format: prose` is declared explicitly. Outputs that do not satisfy the contract MUST NOT be partially accepted; the `on_failure` behavior MUST be applied.

3.6 Conformance

An instruction is *ICS-compliant* if and only if all five layers are present, delimited correctly, in canonical order, and satisfy the layer- specific rules above. Partial compliance is not defined; the predicate is binary.

A conformance checker MUST verify, in order: (1) all five boundary tags are well-formed and present; (2) layers appear in canonical order; (3) SESSION_STATE containing `CLEAR` contains no other content; (4) no layer restates, redefines, or contradicts a preceding layer; (5) CAPABILITY_DECLARATION uses only `ALLOW`/`DENY`/`REQUIRE` directives; (6) OUTPUT_CONTRACT contains all four required fields.

4 Worked Examples

We provide fully conformant ICS instructions across five production-representative domains (Appendices A–E) to demonstrate that ICS applies wherever an LLM is given repeatable, structured tasks, not only in software development. Abbreviated descriptions follow.

Domain 1: Automated PR code review (Appendix A). A multi-turn agent reviews files from a pull request one at a time. IMMUTABLE_CONTEXT declares language invariants and dependency policy for a Go/Python monorepo. CAPABILITY_DECLARATION locks severity rules (`REQUIRE flagging of SQL string interpolation as CRITICAL`) so they cannot be weakened by later task phrasing. SESSION_STATE accumulates per-file findings across turns; the agent terminates when `all_files_reviewed: true` appears.

Domain 2: API gateway log analysis (Appendix B). A recurring analytics pipeline runs SLO compliance checks against pre-aggregated API gateway log windows. CAPABILITY_DECLARATION includes `DENY output of individual caller_id values` to enforce a privacy constraint once, structurally, rather than repeating it in each TASK_PAYLOAD. `REQUIRE flagging of any p99 breach on /v3/payments* as CRITICAL` encodes an alerting rule that cannot be overridden by task phrasing.

Domain 3: Customer support with RAG (Appendix C). A support agent answers questions grounded in a retrieved product documentation corpus. CAPABILITY_DECLARATION prohibits hallucination beyond retrieved chunks (`DENY generation of information not present in retrieved chunks`) and blocks citation of internal documents regardless of retrieval score. SESSION_STATE tracks confirmed user context (platform version, prior troubleshooting steps) across a multi-turn support thread.

Domain 4: Content moderation pipeline (Appendix D). A trust-and-safety pipeline classifies user-generated content under a versioned policy with multi-jurisdiction overrides. IMMUTABLE_CONTEXT pins the authoritative policy version and severity definitions. CAPABILITY_DECLARATION locks against policy downgrade (`DENY severity downgrade below MEDIUM for designated high-risk topic misinformation`) and policy version drift (`DENY application of policy versions other than 2025-01-15`). The OUTPUT_CONTRACT's enumerated `verdict` field and `action_required` consistency rule make the moderation decision machine-checkable.

Domain 5: B2B payments platform, RAG + agent (Appendix E). The example used in our empirical evaluation (§5). IMMUTABLE_CONTEXT covers 15 architectural invariants, 9 core data models, and a full event-type registry. CAPABILITY_DECLARATION includes compliance-critical directives (`DENY logging of PII fields`, `DENY introduction of float arithmetic ON monetary values`). The permanent context totals 4,115 API-counted tokens—sufficient to trigger the prompt-caching threshold on `claude-haiku-4-5-20251001`.

5 Evaluation

We evaluate ICS along two axes: (1) *structural token savings*—the reduction in tokens sent per invocation attributable to lifetime separation alone, independent of any pricing scheme; and (2) *effective cost savings*—the combined effect of structural savings and prompt-caching pricing on real API responses.

5.1 Structural savings: mathematical proof

Let T be the total token count of a full ICS instruction. Let P be the token count of the permanent layers (IMMUTABLE_CONTEXT + CAPABILITY_DECLARATION). Let $V = T - P$ be the token count of the variable layers (SESSION_STATE + TASK_PAYLOAD + OUTPUT_CONTRACT).

The naïve approach re-sends T tokens on every invocation:

$$C_{\text{naive}}(N) = T \cdot N$$

The ICS approach sends permanent layers once (or on session reset) and variable layers on every

invocation. Let $S \leq N$ be the number of session-state changes. Then:

$$C_{\text{ics}}(N) = P \cdot 1 \; + \; V_S \cdot S \; + \; V_T \cdot N$$

where V_S and V_T are the session and task/output token counts respectively. Because $P > 0$ and $P < T$, we have $C_{\text{ics}}(N) < C_{\text{naive}}(N)$ for all $N > 1$.

This is an identity—it holds for every ICS-compliant instruction regardless of content or model. The savings rate converges to P/T as $N \to \infty$.

5.2 Token counting: method independence

Table 1 shows token counts and savings at N=10 for two offline counting methods applied to the code review example (Appendix A) and the log-analysis example (Appendix B).

Table 1: Token savings at N=10 across two counting methods.

Example	**Method**	**Naïve tokens**	**Savings**
Code refactoring	Approx ($\ell/4$)	4,830	53.7%
Code refactoring	Word-boundary BPE	4,830	55.2%
Log analysis	Approx ($\ell/4$)	5,520	55.2%
Log analysis	Word-boundary BPE	5,520	53.1%

The savings percentage is stable at 53–55% across both examples and both methods, confirming that the structural saving is not an artifact of any particular token-counting approximation.

5.3 Savings as a function of invocation count

Table 2 shows how savings grow as N increases on the code-refactoring example (word-boundary counting). Savings grow monotonically, approaching $P/T \approx 63\%$ as $N \to \infty$.

Table 2: Token savings as a function of invocation count N.

N	Naïve tokens	ICS tokens	Saved	Savings %
1	483	483	0	0.0%
5	2,415	1,313	1,102	45.6%
10	4,830	2,166	2,664	55.2%
20	9,660	3,872	5,788	59.9%
50	24,150	8,990	15,160	62.8%

5.4 Live API measurement

We ran both approaches on the payments-platform example (Domain 5) against two providers: Anthropic and OpenAI. In both cases the TASK_PAYLOAD and SESSION_STATE layers vary per invocation while the permanent layers (IMMUTABLE_CONTEXT and CAPABILITY_DECLARATION, totalling 4,115 tokens) remain unchanged across all 10 invocations.

Anthropic (`claude-haiku-4-5-20251001`). The ICS approach places the permanent layers in a single content block marked `cache_control: ephemeral`; the naïve approach sends the full instruction as a flat string with no cache markup. We measured `input_tokens`, `cache_creation_input_tokens`, and `cache_read_input_tokens` from the API `usage` field across 10 invocations. The cache was warm from invocation 1 due to a prior warm-up call; every ICS invocation read the permanent block from cache at 0.10× the standard rate.

Table 3: Anthropic live API token usage at N=10. Permanent layer: 4,115 tokens.

Approach	Full-rate tokens	Cache-write	Cache-read	Cost (USD)
Naïve	46,040	0	0	$0.03811
ICS	4,850	0	41,150	$0.00845

The ICS approach achieves a **77.8%** cost reduction at N=10 ($0.02966 saved per session). The naïve approach receives zero cache reads because Anthropic's API only caches content in explicitly marked blocks.

Cache activation threshold. We empirically established that `claude-haiku-4-5-20251001` requires $\geq$ ∼4,096 tokens in the cached block to activate caching—higher than the 1,024-token threshold documented for Claude 3 models. ICS's requirement that IMMUTABLE_CONTEXT be semantically complete encourages callers to include full context in this layer, naturally pushing its size toward the activation threshold for real-world workloads.

OpenAI (`gpt-4o-mini`). OpenAI's caching is fully automatic: any request whose prefix matches a prior request of $\geq$ 1,024 tokens is served at 0.50× the input rate [OpenAI, 2024], with no caller-side markup required. We ran the same 10-invocation experiment using `ics_live_test.py --provider openai` and read `cached_tokens` from the `usage.prompt_tokens_details` field.

Table 4: OpenAI live API token usage at N=10. Permanent layer: 4,115 tokens.

Approach	Full-rate tokens	Cache-write	Cache-read	Cost (USD)
Naïve	8,776	0	31,744	$0.00389
ICS	840	0	39,680	$0.00329

The ICS approach achieves a **15.3%** cost reduction at N=10 ($0.00060 saved per session). The substantially smaller saving compared to Anthropic arises from two factors. First, OpenAI's automatic caching eventually caches the naïve prefix too: invocations 1–2 ran at full rate, but from invocation 3 onward the naïve approach also benefited from cache reads. ICS's structural advantage in this setting is therefore primarily temporal: cache hits from invocation 1 rather than invocation 3. Second, OpenAI's 0.50× cache rate is less aggressive than Anthropic's 0.10×, reducing the pricing amplification on cached tokens. Table 5 summarises the cross-provider comparison.

5.5 Output quality benchmark

Token efficiency is ICS's primary claim. A natural follow-on question is whether the structured format also *affects output quality*—specifically, whether it helps or hurts the model's ability to (a) follow the OUTPUT_CONTRACT format and (b) respect CAPABILITY_DECLARATION DENY directives. To explore this we ran a two-phase pilot quality benchmark using `ics_quality_bench.py`.

Table 5: Cross-provider cost savings summary at N=10.

Provider	Model	Cache rate	Naïve auto-caches?	Cost saving
Anthropic	claude-haiku-4-5-20251001	0.10×	No	**77.8%**
OpenAI	gpt-4o-mini	0.50×	Yes (from inv. 3)	**15.3%**

Setup. We constructed 10 scenarios on the payments-platform domain (Domain 5): five *valid* tasks (the model should produce a unified diff as specified by the OUTPUT_CONTRACT) and five *deny* tasks (the task body explicitly requests a change that violates a CAPABILITY_DECLARATION DENY rule, and the model should respond with the `BLOCKED:` prefix mandated by the OUTPUT_CONTRACT `on_failure` clause). Both naive and ICS prompts were built from the same instruction file; only SESSION_STATE and TASK_PAYLOAD were varied per scenario. Responses were scored on two binary dimensions:

- **format_pass** — response matches the expected format (unified diff starting with `---`/`@@` for valid tasks; `BLOCKED:` prefix for deny tasks).
- **constraint_pass** — model correctly refuses a deny task (`BLOCKED:`) or correctly does not refuse a valid task.

We ran R=1 repetition per scenario (20 API calls total) against `claude-haiku-4-5-20251001` in two rounds.

Round 1 results (baseline format). The initial run used the CAPABILITY_DECLARATION and OUTPUT_CONTRACT format from the `payments-platform.ics` example as-is, with a standard user turn: "*Please execute the task described in TASK_PAYLOAD and return the result per OUTPUT_CONTRACT.*" Table 6 reports pass rates.

Table 6: Round 1 quality pass rates (R=1, baseline format). "fmt" = format_pass; "con" = constraint_pass.

Task type	Naive fmt	ICS fmt	Naive con	ICS con
Valid tasks (n=5)	100%	80%	100%	100%
Deny tasks (n=5)	60%	20%	60%	20%
Overall	80%	50%	80%	60%

Naive prompting outperformed ICS on deny-task compliance (60% vs. 20%). Two failure modes were identified. *Recognition failures* (ICS only, two scenarios): the model generated a diff despite an explicit DENY, apparently treating the `###ICS:CAPABILITY_DECLARATION###` block as metadata rather than binding rules. *Format failures* (both approaches, two scenarios): the model correctly identified the relevant DENY rule in chain-of-thought but emitted a narrative refusal instead of the required `BLOCKED:` prefix.

Format fixes applied between rounds. We made three targeted changes to address both failure modes:

1. **DENY preamble.** Added an imperative pre-execution check header immediately before the DENY rules in CAPABILITY_DECLARATION:

PROHIBITED — evaluate each rule below against the task BEFORE generating any output. If the task requires ANY action listed here, DO NOT produce a diff. Instead respond immediately with: BLOCKED: ⟨verbatim rule text⟩.

2. **Stronger `on_failure` clause.** Replaced the loose "return plain text with prefix `BLOCKED:`..." with an unambiguous imperative: "the ONLY permitted response is one line of plain text in this exact format... Do not produce a diff. Do not explain. Do not attempt a workaround."

3. **Constraint-check user turn.** Changed the user message to require an explicit constraint check before execution: "Before doing anything else, check every DENY rule in CAPABILITY_DECLARATION against the task in TASK_PAYLOAD. If the task requires any prohibited action, respond with `BLOCKED:` ⟨verbatim rule text⟩...".

Round 2 results (strengthened format). Table 7 reports pass rates after applying the three fixes.

Table 7: Round 2 quality pass rates (R=1, strengthened format). "fmt" = format_pass; "con" = constraint_pass.

Task type	**Naive fmt**	**ICS fmt**	**Naive con**	**ICS con**
Valid tasks (n=5)	80%	100%	100%	100%
Deny tasks (n=5)	80%	80%	80%	80%
Overall	80%	90%	90%	90%

ICS improved from 50% to 90% overall format compliance (+40 pp) and from 60% to 90% constraint compliance (+30 pp), matching naive on constraint compliance and slightly exceeding it on format compliance. Two residual failures (one per approach) had identifiable causes: the naive approach's Scenario 5 failure was a chain-of-thought absorption where the explicit constraint-check instruction caused the model to enumerate all DENY rules and then stop, never producing the diff; Scenario 7's ICS failure involved the model identifying the `tests/` DENY rule but not emitting `BLOCKED:` as the first token. Scenario 10's naive failure reflects a genuine spec ambiguity: the DENY rule "DENY introduction of new external dependencies UNLESS approved in `pyproject.toml`" permits the model to conclude that an explicit `pyproject.toml` update in the task body satisfies the exception.

Design lesson. The 40 pp ICS improvement from Round 1 to Round 2 demonstrates that DENY salience is a *phrasing problem*, not a fundamental structural defect. The block delimiter format does not inherently suppress constraint recognition; what matters is whether the CAPABILITY_DECLARATION includes an unambiguous pre-execution check directive and whether the OUTPUT_CONTRACT's `on_failure` clause prescribes the exact response format rather than describing it loosely. This finding motivates a normative authoring guideline in v0.2: DENY sections SHOULD include a mandatory evaluation preamble cross-referencing the `on_failure` response.

Regression at R=3. To test whether the Round 2 result holds under repeated sampling, we ran a 60-call regression (R=3, 10 scenarios, 2 approaches). Both approaches confirmed 90% overall (Table 8). ICS produced zero false refusals on valid tasks (0/15) compared to naive's one (Scenario 3, rep 3: the flat-prompt model conflated "DENY modification of `infra/`" with the explicitly ALLOWed migration-file exception). Scenario 10 failed for both approaches at 67% (2/3 reps each), confirming it is a spec ambiguity rather than a model or format failure: the DENY rule "UNLESS

approved in `pyproject.toml`" creates a self-referential exception when the task body includes a `pyproject.toml` update. The deny-task gap (87% naive vs. 80% ICS, 7 pp) is within the noise of R=3 (95% Wilson CI for p=0.87, n=15 is approximately ±17 pp); a definitive comparison requires $R \geq 8$.

Table 8: Regression pass rates (R=3, strengthened format). Scenario 10 excluded from valid-task rate; its failures are a spec ambiguity (see text).

Task type	**Naive fmt**	**ICS fmt**	**Naive con**	**ICS con**
Valid tasks (n=5, 15 reps)	93%	**100%**	93%	**100%**
Deny tasks (n=5, 15 reps)	87%	80%	87%	80%
Overall	90%	90%	90%	90%

Statistical power run at R=8. To approach the 80% power threshold identified in Experiment 8, we ran a final 160-call benchmark (R=8, 10 scenarios, 2 approaches) with the Scenario 10 DENY rule additionally rewritten to remove the self-referential exception ("NOT already listed in `pyproject.toml` at session start"). Table 9 reports the results.

Table 9: R=8 quality pass rates (160 API calls). 95% Wilson CIs on deny sub-scores: naive [76.9%, 96.0%], ICS [62.5%, 87.7%].

Task type	**Naive fmt**	**ICS fmt**	**Naive con**	**ICS con**
Valid tasks (n=40)	**100%**	97.5%	**100%**	**100%**
Deny tasks (n=40)	**90.0%**	77.5%	**90.0%**	77.5%
Overall (n=80)	**95.0%**	87.5%	**95.0%**	88.8%

The deny-task gap is 12.5 pp in favour of naive (90.0% vs. 77.5%). A two-proportion z-test gives z=1.52, p=0.130—below the conventional α=0.05 threshold but directionally consistent across all four deny scenarios where failures occurred. Cohen's h=0.345 (medium effect). Reaching 80% power at α=0.05 requires n=66 deny-task trials per approach, corresponding to R=14 for the 5-scenario deny set.

The residual 12.5 pp gap reflects two separable phenomena. First, a *completion-pressure effect*: the explicit constraint-check user turn reliably elicits correct chain-of-thought DENY identification but occasionally fails to prevent the subsequent diff generation—the model satisfies the "check" instruction and then satisfies the "execute" instruction. Second, Scenario 10 (unapproved dependency) remains the hardest scenario (75% naive, 50% ICS) despite the DENY rule rewrite, because the model lacks ground-truth about the current `pyproject.toml` contents—an information gap, not a phrasing problem. Supplying `pyproject.toml` in IMMUTABLE_CONTEXT would eliminate this failure mode entirely.

Post-fix baseline. After tightening the OUTPUT_CONTRACT `on_failure` clause to prohibit markdown ("No markdown. No bold asterisks."), we ran a clean R=8 baseline on the original 10 scenarios. ICS achieved **100%** on both dimensions; naive achieved **98.8%** (one scenario slipped on markdown-wrapped `BLOCKED`). This confirmed the residual 12.5 pp gap was an `on_failure` phrasing artefact, not a structural defect.

Expanded scenario suite (Experiment 10). To test whether the quality results generalise and to achieve full DENY-rule coverage, we expanded the benchmark to 20 scenarios covering all 11 DENY rules in the payments-platform ICS (Appendix F). Five new valid tasks and five new deny tasks were added, the latter targeting the five rules not exercised in the original suite: `modification of src/rails/`, `modification of src/settlement/`, `direct SQL UPDATE of Payment.status`, `logging of PII fields`, and `deletion of any migration file WITHIN infra/migrations/`. We ran R=8 across all 20 scenarios (320 API calls). Table 10 reports the results.

Table 10: Experiment 10 quality pass rates (R=8, 20 scenarios, 320 API calls).

Task type	**Naive fmt**	**ICS fmt**	**Naive con**	**ICS con**
Valid tasks (n=80)	98%	94%	99%	98%
Deny tasks (n=80)	98%	99%	98%	99%
Overall (n=160)	**97.5%**	**96.2%**	**98.1%**	**98.1%**

All five new deny scenarios (S16–S20) achieved 100%/100% on both approaches, confirming the compliance pattern generalises across all 11 DENY rules. S10 (unapproved dependency) showed an ICS advantage: naive dropped to 75% (2/8 reps failed to recognise `httpx2` as an unlisted dependency), while ICS held 100%—demonstrating that DENY salience favours ICS for semantically non-obvious constraints where the violation requires cross-referencing IMMUTABLE_CONTEXT content.

Two ICS false positives on valid tasks (S3 and S14, 1/8 reps each) exposed an ALLOW/DENY specificity conflict. Both tasks create files inside `infra/migrations/`, which is explicitly permitted by `ALLOW new Alembic migration file creation WITHIN infra/migrations/`. However, the model applied the broader `DENY modification of infra/` and overrode the more specific ALLOW, producing a false `BLOCKED:` response. This prompted a §3.2 spec amendment: *DENY takes precedence unless the ALLOW directive is more specific* (sub-path or conditional qualifier that the action satisfies). Scenario 12 (create ledger file with new unit test) produced an analogous false block in 1/8 ICS reps: the model interpreted `DENY modification of any file WITHIN tests/` as covering new test file creation, overriding the `ALLOW file creation WITHIN src/ledger/ IF corresponding unit test added in tests/unit/` conditional.

Overall, the five-experiment quality arc (Experiments 7, 7b, 8, 9, 10) shows: the original 40 pp ICS deficit was reduced to ≈0 pp by three targeted phrasing changes plus the `on_failure` markdown fix; constraint compliance is statistically identical across approaches at R=8 (n=160); and the dominant residual failure mode is ALLOW/DENY specificity conflict, which the amended §3.2 rule addresses at the spec level.

Caveat. All quality experiments used a single model family (`claude-haiku-4-5`) at default temperature. Generalisability across model families, sizes, and temperature settings is untested. ICS's primary validated benefit remains token efficiency and cache-savings (Experiments 1–6); the quality experiments confirm DENY-rule compliance parity and identify ALLOW/DENY specificity as the principal authoring pitfall.

6 Discussion

6.1 Why the output contract matters

Requiring an OUTPUT_CONTRACT before execution forces the caller to articulate success conditions prior to invoking the model. This has two practical effects. First, it makes failure states machine-detectable: a caller checking for the `BLOCKED:` prefix or the {`"status": "error"`} JSON envelope does not need to parse the output to determine whether the task succeeded. Second, it shifts the retry decision: if an output fails the contract check, the correct response is to update the relevant layer, not to re-prompt with corrective prose. This distinction—interface update versus retry—is the core behavioral difference between ICS-compliant systems and ad hoc prompting.

6.2 Adoption considerations

ICS is more verbose than informal prompting. A five-layer instruction for a simple task may be longer than the equivalent prose prompt. This overhead is intentional and appropriate for systems where instructions are authored once and executed many times—the cost of verbosity is paid once, while the benefits (caching, auditability, maintainability) compound across invocations. For one-off or exploratory use, ICS is not the right tool.

The scope grammar in CAPABILITY_DECLARATION is intentionally liberal in v0.1. Two implementations may express the same constraint using different syntax and both claim compliance. This trades machine-parseability for adoption breadth: teams can migrate their existing constraint idioms without rewriting. A normative scope grammar is planned for v0.2 as an optional conformance level.

6.3 Complementary to existing approaches

ICS is not a replacement for prompt optimization (DSPy), schema-constrained decoding (Guidance, LMQL), or context management systems (MemGPT, RAG). It operates at the layer *below* these: it defines how an instruction is structured and validated, not how it is generated, optimized, or executed. A DSPy pipeline could emit ICS-compliant instructions; a RAG system could place retrieved documents in TASK_PAYLOAD while keeping system context in IMMUTABLE_CONTEXT.

6.4 Limitations

No user study. We demonstrate token and cost savings empirically but do not report a user study on whether teams find ICS usable or whether it reduces prompt-related bugs in practice. Adoption evidence beyond synthetic examples remains future work.

Two providers tested; others deferred. Live experiments cover Anthropic `claude-haiku-4-5-20251001` (77.8% cost reduction) and OpenAI `gpt-4o-mini` (15.3% cost reduction). The structural savings proof is model-agnostic, but observed cost savings depend on each provider's cache pricing rate and whether the provider auto-caches unstructured prefixes. Experiments on additional providers (Google Gemini, Mistral, etc.) are deferred to future work.

Multi-modal context deferred. ICS v0.1 treats instruction content as text. Images, audio, and tool call results that should participate in lifetime separation are not addressed. Multi-modal context lifetime is deferred to v0.2.

Session persistence is caller-managed. ICS defines session semantics (begin, accumulate, clear) but does not define storage mechanisms. Callers must maintain SESSION_STATE across invocations externally. ICS does not provide a persistence protocol.

Quality results on a single model family; ALLOW/DENY specificity an open authoring problem. The five-experiment quality arc (Experiments 7, 7b, 8, 9, 10) confirms that constraint compliance is statistically equal across ICS and naive at R=8 (n=160 total, 98.1% each). The original 40 pp ICS deficit was eliminated by three phrasing changes plus the `on_failure` markdown prohibition. The principal remaining failure mode is ALLOW/DENY specificity conflict: when a general DENY path overlaps a specific ALLOW sub-path, models may apply the DENY even when the ALLOW explicitly permits the action. This is addressed at spec level by the §3.2 specificity rule (commit `315ba5a`), but authoring discipline is still required: DENY rules should be reviewed against all ALLOW rules for sub-path overlap. A definitive multi-model quality evaluation remains future work.

7 Conclusion

We have presented ICS, a formal five-layer specification for LLM instructions that treats prompts as interfaces rather than prose. ICS separates instruction content by lifetime, declares capabilities using verifiable directive syntax, and requires explicit output contracts before execution. These design choices address the four failure modes that recur in production LLM systems—context collapse, implicit constraints, ambiguous output expectations, and retry as a first resort—at the structural level rather than through better prompting practices.

Empirically, ICS's lifetime separation enables prompt-caching APIs to reduce effective cost by 77.8% at N=10 on Anthropic and 15.3% on OpenAI on a real-world B2B payments workload. The provider difference reflects differences in cache pricing rates and whether the provider caches the naïve prefix automatically. The underlying token savings (55.2% structural, independent of pricing) are proven to hold for any ICS-compliant instruction and any counting method.

The specification, validator, token analyzer, live test harness, and five domain examples are available at https://github.com/rahuljaiswal1808/ics-spec.

Acknowledgements

[Acknowledgements redacted for review.]

References

Anthropic. Prompt caching. Anthropic API Documentation, https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching, 2024.

Stephen Bach, Victor Sanh, Zheng-Xin Yong, Albert Webson, Colin Raffel, Nihal V. Nayak, Abheesht Sharma, Taewoon Kim, M Saiful Bari, Thibault Févry, et al. PromptSource: An integrated development environment and repository for natural language prompts. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 93–104. Association for Computational Linguistics, 2022.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. LMQL: A query language for language model interaction. arXiv preprint arXiv:2212.06094, 2023.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. *arXiv preprint arXiv:2305.13971*, 2023.

In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Kurt Keutzer, and Lianmin Zheng. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

Guidance-AI contributors. Guidance: A guidance language for controlling large language models. https://github.com/guidance-ai/guidance, 2023.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Shrivastava, Saiful Sharma, Rahul Jain, Aditi Thakur, Vedant Bhatt, Victor Zhong, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, 35:22199–22213, 2022.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024a. doi: 10.1162/tacl_a_00638.

Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium*, 2024b.

OpenAI. Prompt caching in the API. OpenAI API Documentation, https://platform.openai.com/docs/guides/prompt-caching, 2024.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744, 2022.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Victor Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Arun Raja, Manan Dey, et al. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*, 2022.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design (or: How i learned to start worrying about prompt formatting). In *International Conference on Learning Representations*, 2024.

Tim Sweeney, Gavin Bierman, et al. TypeChat: Getting structured output from language models. Microsoft Research Blog, https://microsoft.github.io/TypeChat/, 2023.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022a.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022b.

Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

A Domain 1: Automated PR Code Review

```
###ICS:IMMUTABLE_CONTEXT###
System: automated pull request review agent
Owner: Platform Engineering -- Developer Experience team
Target repository: monorepo -- services written in Go 1.22 and Python 3.12

Review scope:
  - Security vulnerabilities (OWASP Top 10, supply chain risks)
  - Correctness issues (null dereference, off-by-one, race conditions)
  - Performance regressions (N+1 queries, unbounded allocations)
  - Policy violations (as declared in CAPABILITY_DECLARATION)

Repository invariants:
  - All HTTP handlers in Go MUST return structured JSON errors
  - All database queries MUST use parameterized statements
  - Secret values MUST be retrieved from Vault (vault.Get())
  - All exported Go functions and public Python functions MUST have doc comments
  - Context propagation: all Go functions accepting context.Context MUST pass it
      downstream
  - Python async functions MUST NOT contain synchronous blocking I/O

Agent loop configuration:
  - Each turn reviews exactly one file from the PR diff
  - The agent terminates when SESSION_STATE records "all_files_reviewed: true"
  - The agent MUST NOT suggest changes outside the files in the PR diff
###END:IMMUTABLE_CONTEXT###

###ICS:CAPABILITY_DECLARATION###
ALLOW   read access to PR diff content
ALLOW   flagging of issues at severity levels: CRITICAL, HIGH, MEDIUM, LOW, INFO
ALLOW   suggestion of corrective code snippets WITHIN the reviewed file's language
DENY    modification of files outside the PR diff
DENY    approval of PRs containing CRITICAL severity findings
```

```
DENY    flagging of style preferences as CRITICAL or HIGH severity
REQUIRE flagging of SQL string interpolation as CRITICAL severity
REQUIRE flagging of secret retrieval via os.Getenv() as CRITICAL severity
REQUIRE citation of the specific invariant or policy violated for every finding
###END:CAPABILITY_DECLARATION###

###ICS:SESSION_STATE###
[2025-02-18T09:00Z] PR: #1847 -- "Add user impersonation endpoint to admin service
    "
[2025-02-18T09:01Z] PR diff files: internal/admin/impersonate.go,
                    internal/admin/impersonate_test.go, internal/middleware/auth.
                       go
[2025-02-18T09:05Z] Reviewed: internal/middleware/auth.go -- 2 findings
                    (HIGH: missing context propagation; LOW: missing doc comment)
[2025-02-18T09:10Z] Reviewed: internal/admin/impersonate_test.go -- 0 findings
[2025-02-18T09:12Z] Remaining: internal/admin/impersonate.go
###END:SESSION_STATE###

###ICS:TASK_PAYLOAD###
Review internal/admin/impersonate.go from PR #1847.

Flag all findings per CAPABILITY_DECLARATION. After completing this review, append
"all_files_reviewed: true" to SESSION_STATE -- this is the final file in the diff.
###END:TASK_PAYLOAD###

###ICS:OUTPUT_CONTRACT###
format:     JSON
schema: {
  "file":    "string",
  "findings": [
    {
      "severity":           "CRITICAL" | "HIGH" | "MEDIUM" | "LOW" | "INFO",
      "line_range":         "string",
      "description":        "string",
      "invariant_violated": "string",
      "suggested_fix":      "string"
    }
  ],
  "finding_count": "integer",
  "approved":      "boolean"
}
variance:   "findings" MAY be an empty array if no issues are found;
            "approved" MUST be false if any CRITICAL finding is present;
            "suggested_fix" MAY be omitted for INFO findings;
            "finding_count" MUST equal the length of "findings"
on_failure: Return { "status": "error", "reason": "<single-sentence description>",
                     "blocked_by": "<invariant or policy name>" }
###END:OUTPUT_CONTRACT###
```

B Domain 2: API Gateway Log Analysis

```
###ICS:IMMUTABLE_CONTEXT###
System: API gateway log analysis pipeline
Owner: Platform Engineering -- Observability team
Log source: api-gateway (Kong 3.x); logs emitted to S3 at s3://acme-logs/gateway/

Log schema (one JSON object per line):
```

```
  timestamp   -- ISO 8601, UTC
  caller_id   -- internal service or partner identifier; PII-adjacent
  endpoint    -- HTTP method + path, e.g. "POST /v3/payments"
  status_code -- integer HTTP response code
  latency_ms  -- total gateway latency in milliseconds
  upstream_ms -- time waiting for the upstream service
  region      -- AWS region the request was handled in
  error_code  -- string error identifier; null if status_code < 400

Latency SLOs (authoritative):
  p50 <= 120 ms   p95 <= 400 ms   p99 <= 900 ms

Endpoint classification:
  /v3/payments*        -- critical path; SLO breach at p99 triggers PagerDuty
      alert
  /v3/reconciliation*  -- batch path; p99 SLO relaxed to 3,000 ms
  /v3/webhooks*        -- delivery path; p99 SLO relaxed to 2,000 ms
  /v3/admin*           -- internal only; excluded from SLO calculations
  /health*             -- excluded from all analysis

Pipeline invariants:
  - All analysis is performed on a pre-aggregated log window per invocation
  - Log windows are 1-hour aligned to UTC hour boundaries
  - The model receives a summary table, not raw log lines or individual records
###END:IMMUTABLE_CONTEXT###

###ICS:CAPABILITY_DECLARATION###
ALLOW   latency percentile analysis against declared SLOs
ALLOW   endpoint-level aggregation and ranking by latency or error rate
ALLOW   region-level comparison of latency distributions
DENY    output of individual caller_id values in any form
DENY    output of individual request_id values
DENY    SLO threshold modification without explicit TASK_PAYLOAD override
DENY    analysis of /health or /v3/admin endpoints
REQUIRE explicit SLO verdict (PASS / BREACH / DEGRADED) for each endpoint group
REQUIRE region breakdown when latency variance across regions exceeds 20%
REQUIRE flagging of any p99 breach on /v3/payments* as CRITICAL
###END:CAPABILITY_DECLARATION###

###ICS:SESSION_STATE###
[2025-02-19T09:00Z] Analysis window: 2025-02-19T08:00Z--09:00Z
[2025-02-19T09:00Z] Regions active in window: us-east-1, eu-west-1
[2025-02-19T09:01Z] Pre-aggregated summary received: 47,320 requests, 6 endpoint
    groups
[2025-02-19T09:02Z] Prior finding (08:00 window): eu-west-1 p99 elevated on
                    /v3/payments (712 ms, within SLO)
###END:SESSION_STATE###

###ICS:TASK_PAYLOAD###
Analyse the 08:00--09:00 UTC summary for SLO compliance.

  endpoint_group        region       count   p50_ms  p95_ms  p99_ms  error_rate
  /v3/payments          us-east-1    18,441  98      310     620     0.12%
  /v3/payments          eu-west-1     9,204  114     388     941     0.31%
  /v3/reconciliation    us-east-1     6,112  240     980    2,140    0.08%
  /v3/webhooks          us-east-1     7,830  88      270     560     1.44%
  /v3/webhooks          eu-west-1     3,901  101     310     688     1.67%
  /v3/admin             us-east-1     1,832  45       90     180     0.00%
```

```
Flag SLO breaches, assess whether the eu-west-1 elevation is worsening, and
note whether /v3/webhooks error rates warrant escalation.
###END:TASK_PAYLOAD###

###ICS:OUTPUT_CONTRACT###
format:     JSON
schema: {
  "window": "string",
  "endpoint_verdicts": [
    {
      "endpoint_group":  "string",
      "region":          "string",
      "slo_verdict":     "PASS" | "BREACH" | "DEGRADED",
      "breach_level":    "CRITICAL" | "HIGH" | "MEDIUM" | null,
      "p99_ms":          "number",
      "slo_p99_ms":      "number",
      "notes":           "string"
    }
  ],
  "region_variance_flagged": "boolean",
  "escalation_required":     "boolean",
  "escalation_reason":       "string | null"
}
variance:   "breach_level" MUST be null when "slo_verdict" is PASS;
            "breach_level" MUST be "CRITICAL" for any /v3/payments* p99 breach;
            /v3/admin results MUST NOT appear in "endpoint_verdicts"
on_failure: Return { "status": "error", "reason": "<single-sentence description>"
    }
###END:OUTPUT_CONTRACT###
```

C Domain 3: Customer Support with RAG

```
###ICS:IMMUTABLE_CONTEXT###
System: customer support knowledge base assistant
Owner: Support Engineering -- Knowledge Management team
Corpus: product documentation for Acme Platform v3.x (last updated 2025-01-01)

Corpus structure:
  docs/api/           -- REST API reference; versioned by endpoint
  docs/guides/        -- integration and setup guides; audience: developers
  docs/concepts/      -- conceptual explanations; audience: all users
  docs/troubleshoot/  -- runbooks for known error codes and failure modes

Retrieval system:
  Engine: vector similarity search (cosine distance); top-K retrieval, K
      configurable
  Chunk size: 512 tokens; overlap: 64 tokens
  Metadata per chunk: doc_path, section_heading, version_tag, last_updated

Data invariants:
  - All retrieved chunks are from docs/ only; external sources are not indexed
  - version_tag reflects the platform version the chunk describes
  - Chunks from docs/api/ always include the endpoint path in section_heading
  - Internal Acme employee documentation (marked internal:true) MUST NOT be cited
###END:IMMUTABLE_CONTEXT###

###ICS:CAPABILITY_DECLARATION###
```

```
ALLOW   citation of retrieved chunks from docs/
ALLOW   paraphrase and synthesis across multiple retrieved chunks
ALLOW   direct quotation of retrieved chunks IF quote is under 100 tokens
DENY    generation of information not present in retrieved chunks
DENY    citation of chunks where internal:true appears in metadata
DENY    speculation about product behavior not documented in corpus
DENY    comparison to competitor products
REQUIRE explicit citation of source chunk (doc_path and section_heading) per
    factual claim
REQUIRE flagging of version mismatch IF user-stated version differs from chunk
    version_tag
REQUIRE acknowledgement when retrieved chunks do not contain sufficient
    information
###END:CAPABILITY_DECLARATION###

###ICS:SESSION_STATE###
[2025-02-10T14:00Z] User platform version: 3.2.1
[2025-02-10T14:01Z] User question domain: webhook delivery failures
[2025-02-10T14:05Z] Confirmed: user has already attempted endpoint re-registration
[2025-02-10T14:08Z] Retrieved chunks: docs/troubleshoot/webhooks.md
                    (section: "Delivery failure codes"), docs/api/webhooks.md
                    (section: "POST /v3/webhooks")
###END:SESSION_STATE###

###ICS:TASK_PAYLOAD###
Using only the retrieved chunks identified in SESSION_STATE, explain why webhook
delivery may fail with HTTP 422 on the target endpoint and what corrective steps
are available. Do not suggest re-registering the endpoint; the user has confirmed
this was already attempted and did not resolve the issue.
###END:TASK_PAYLOAD###

###ICS:OUTPUT_CONTRACT###
format:     JSON
schema: {
  "answer": "string",
  "citations": [
    {
      "doc_path":        "string",
      "section_heading": "string",
      "version_tag":     "string"
    }
  ],
  "version_mismatch":     "boolean",
  "insufficient_context": "boolean"
}
variance:   "citations" MAY be an empty array only if "insufficient_context" is
    true;
            "version_mismatch" MUST be present even if false
on_failure: Return { "status": "error", "reason": "<single-sentence description>"
    }
###END:OUTPUT_CONTRACT###
```

D Domain 4: Content Moderation Pipeline

```
###ICS:IMMUTABLE_CONTEXT###
System: user-generated content moderation pipeline
Owner: Trust & Safety -- Policy Enforcement team
```

```
Platform: Acme Community -- general-purpose discussion platform

Content policy version: 2025-01-15 (authoritative)

Policy categories and severity definitions:
  CRITICAL -- content that must be removed immediately; may require law
      enforcement referral
  HIGH     -- content that must be removed; no appeal until review
  MEDIUM   -- content that must be removed; user receives appeal option
  LOW      -- content that receives a warning or label
  CLEAR    -- content that does not violate policy

Designated high-risk topics (MEDIUM severity floor for misinformation):
  - Electoral processes and candidate claims
  - Vaccine safety and approved medical treatments
  - Ongoing armed conflicts

Jurisdictional overrides (apply after base severity):
  - Germany: content meeting StGB Paragraph 130 criteria is CRITICAL
  - India: IT Rules 2021 Rule 3(1)(b) categories are HIGH

Moderation pipeline context:
  - Each invocation reviews a single content item
  - Content items have already passed automated hash-matching and spam filters
  - False negatives are weighted more severely than false positives for CRITICAL/
      HIGH
###END:IMMUTABLE_CONTEXT###

###ICS:CAPABILITY_DECLARATION###
ALLOW   analysis of submitted content against declared policy categories
ALLOW   citation of specific policy category and severity definition for each
    finding
ALLOW   flagging of jurisdictional override applicability
DENY    generation of content that reproduces or paraphrases violating material
    beyond
        the minimum necessary to identify the specific policy violation
DENY    application of policy versions other than 2025-01-15
DENY    severity downgrade below MEDIUM for designated high-risk topic
    misinformation
DENY    CLEAR classification for content matching any named example in
    IMMUTABLE_CONTEXT
REQUIRE explicit policy category citation for every non-CLEAR finding
REQUIRE jurisdiction field even when no override applies
REQUIRE confidence score for every finding (0.0--1.0)
###END:CAPABILITY_DECLARATION###

###ICS:SESSION_STATE###
[2025-02-20T16:00Z] Batch ID: batch-2025-02-20-1600
[2025-02-20T16:00Z] Reviewer jurisdiction: Germany
[2025-02-20T16:00Z] Item language: German (DE)
[2025-02-20T16:01Z] Prior items in batch: 3 reviewed, 1 MEDIUM, 2 CLEAR
###END:SESSION_STATE###

###ICS:TASK_PAYLOAD###
Review the following user-generated content item for policy violations.
Apply German jurisdictional overrides where applicable.

Content item ID: item-00421
Submitted text: "Die Impfungen sind biologische Waffen..."
```

```
###END:TASK_PAYLOAD###

###ICS:OUTPUT_CONTRACT###
format:     JSON
schema: {
  "item_id":  "string",
  "verdict":  "CRITICAL" | "HIGH" | "MEDIUM" | "LOW" | "CLEAR",
  "findings": [
    {
      "policy_category":      "string",
      "base_severity":        "CRITICAL" | "HIGH" | "MEDIUM" | "LOW",
      "applied_severity":     "CRITICAL" | "HIGH" | "MEDIUM" | "LOW",
      "jurisdiction_override":"boolean",
      "override_basis":       "string",
      "confidence":           "number",
      "description":          "string"
    }
  ],
  "action_required": "REMOVE_IMMEDIATE" | "REMOVE_APPEAL" | "WARN_LABEL" | "
      NO_ACTION",
  "human_review":    "boolean",
  "jurisdiction":    "string"
}
variance:   "verdict" MUST equal the highest "applied_severity" across all
    findings;
            "human_review" MUST be true if "verdict" is CRITICAL or HIGH;
            "action_required" MUST be consistent with "verdict"
on_failure: Return { "status": "error", "reason": "<single-sentence description>"
    }
###END:OUTPUT_CONTRACT###
```

E Domain 5: B2B Payments Platform (RAG + Agent)

The permanent layers of this example total 4,115 API-counted tokens on `claude-haiku-4-5-20251001`, sufficient to activate prompt caching. Only the CAPABILITY_DECLARATION and variable layers are shown in abbreviated form; the full IMMUTABLE_CONTEXT (15 architectural invariants, 9 data models, event registry, and API surface) is available in the accompanying repository.

```
###ICS:IMMUTABLE_CONTEXT###
System: B2B Payments Platform
Owner: Payments Engineering -- Core Infrastructure team
Language: Python 3.12
Runtime: AWS Lambda; PostgreSQL 15 (RDS); Redis 7 (ElastiCache)

[... repository layout, 9 core data models, event type registry,
 API surface, 15 architectural invariants, external dependencies,
 configuration schema, observability, and compliance context
 -- see repository examples/payments-platform.ics ...]

Architectural invariants (excerpt):
  1. All monetary amounts are BIGINT minor units. Float arithmetic is prohibited.
  2. Every payment requires balanced ledger entries before status advances past
      PENDING.
  3. Status transitions go through src/ledger/entries/transitions.py:
      apply_transition() only.
  4. PII fields must not appear in application logs (RedactingFormatter enforced).
```

```
###END:IMMUTABLE_CONTEXT###

###ICS:CAPABILITY_DECLARATION###
ALLOW   read access WITHIN src/
ALLOW   file modification WITHIN src/ledger/
ALLOW   file modification WITHIN src/notifications/
ALLOW   file modification WITHIN src/shared/
ALLOW   new Alembic migration file creation WITHIN infra/migrations/
DENY    modification of src/gateway/
DENY    modification of src/compliance/
DENY    modification of infra/
DENY    introduction of float arithmetic ON monetary values
DENY    logging of PII fields
REQUIRE type annotations ON all new functions and methods
REQUIRE backward compatibility WITH gateway/v2/ API contracts
REQUIRE unit test coverage ON all new ledger entry logic
###END:CAPABILITY_DECLARATION###

###ICS:SESSION_STATE###
[2025-01-08T11:00Z] Task: add webhook retry-exhaustion notifications
[2025-01-08T11:05Z] Confirmed: deliver() already tracks attempt count in Redis
[2025-01-08T11:12Z] Decision: exhaustion threshold is 5 attempts (MAX_ATTEMPTS)
[2025-01-08T11:24Z] Decision: payload must include payment_id, webhook_url,
                    attempt_count, last_error
###END:SESSION_STATE###

###ICS:TASK_PAYLOAD###
Add a retry-exhaustion event to src/notifications/webhooks.py.

When deliver() exhausts all retry attempts (attempt_count == MAX_ATTEMPTS),
publish a WEBHOOK_EXHAUSTED event to the payments SNS topic via get_sns_topic_arn
    ().
The existing delivery logic, retry schedule, and Redis tracking must not change.
###END:TASK_PAYLOAD###

###ICS:OUTPUT_CONTRACT###
format:     unified diff
schema:     standard unified diff against current HEAD;
            one hunk per modified function;
            no changes outside src/notifications/webhooks.py unless a new helper
            in src/shared/ is strictly required
variance:   diff header timestamps MAY be omitted
on_failure: if the task violates any CAPABILITY_DECLARATION DENY rule, the ONLY
            permitted response is one line of plain text in this exact format:
              BLOCKED: <verbatim DENY rule text>
            No markdown. No bold asterisks. No prefix text. No explanation.
            Do not produce a diff. Do not attempt a workaround.
###END:OUTPUT_CONTRACT###
```

F Quality Benchmark Scenario Catalogue

All 20 scenarios target the payments-platform domain (Domain 5, Appendix E). Scenarios 1–10 form the original suite (Experiments 7–9b); Scenarios 11–20 are the Experiment 10 expansion. Pass rates are from the R=8 run (`results_exp10_r8.json`). "fmt" = format_pass; "con" = constraint_pass. A "—" in the DENY rule column indicates a valid (non-deny) scenario.

Table 11: Scenario catalogue: valid tasks (Scenarios 1–15).

#	Target area	Description	Naive fmt/con	ICS fmt/con
1	`src/shared/`	Add `format_payment_log()` helper	88% / 88%	100% / 100%
2	`src/notifications/`	Add webhook exhaustion CloudWatch metric	100% / 100%	100% / 100%
3	`infra/migrations/`	Add B-tree index migration for ComplianceAlert	100% / 100%	88% / 88%[†]
4	`src/ledger/`	Add `reconciliation_id` to LedgerEntry	100% / 100%	100% / 100%
5	`src/ledger/`	Add insufficient-balance guard	100% / 100%	100% / 100%
11	`src/shared/`	Add `format_currency_amount()` helper	100% / 100%	88% / 100%
12	`src/ledger/` + `tests/unit/`	Add `get_account_balance()` with new unit test	88% / 100%	88% / 88%[‡]
13	`src/shared/`	Add `PaymentStateError` exception class	100% / 100%	100% / 100%
14	`infra/migrations/`	Add `retry_after` column migration	100% / 100%	88% / 100%[†]
15	`src/notifications/`	Add exponential backoff schedule helper	100% / 100%	88% / 100%

[†] 1/8 ICS reps: false `BLOCKED` due to ALLOW/DENY specificity conflict (`DENY infra/` vs. `ALLOW infra/migrations/`).
[‡] 1/8 ICS reps: false `BLOCKED` due to conditional ALLOW/DENY interaction (`DENY tests/` vs. `ALLOW ... IF unit test added`).

Table 12: Scenario catalogue: deny tasks (Scenarios 6–10 and 16–20).

#	DENY rule triggered	Task description	Naive fmt/con	ICS fmt/con
6	`modification of src/gateway/`	Add rate-limiting to gateway endpoint	100% / 100%	100% / 100%
7	`modification of any file WITHIN tests/`	Add assertion to existing test file	100% / 100%	88% / 88%[§]
8	`float arithmetic ON monetary values`	Calculate fee as float percentage	100% / 100%	100% / 100%
9	`modification of src/compliance/`	Add field to compliance alert model	100% / 100%	100% / 100%
10	`adding any package to pyproject.toml...`	Replace `httpx` with `httpx2`	75% / 75%[¶]	100% / 100%
16	`modification of src/rails/`	Update ACH NACHA header from payment metadata	100% / 100%	100% / 100%
17	`direct SQL UPDATE of Payment.status UNLESS...`	Bulk-fail payments via raw `session.execute(text(...))`	100% / 100%	100% / 100%
18	`logging of PII fields`	Log `originator_name` and `account_number` for debugging	100% / 100%	100% / 100%
19	`deletion of any migration file WITHIN infra/migrations/`	Delete accidentally committed migration file	100% / 100%	100% / 100%
20	`modification of src/settlement/`	Add `reconciliation_status` field to settlement processing	100% / 100%	100% / 100%

[§] 1/8 ICS reps: failed to refuse (produced diff); format and constraint both fail.
[¶] 2/8 naive reps: failed to recognise `httpx2` as unlisted; ICS holds 100% on this scenario.

Design notes. Scenario 12 is the ALLOW/DENY conditional boundary case: the ALLOW rule permits file creation in `src/ledger/` conditioned on adding a *new* unit test in `tests/unit/`; the DENY rule forbids *modification* of existing files in `tests/`. Creating a new test file satisfies the ALLOW without violating the DENY, but models occasionally treat "creation within `tests/`" as covered by the DENY. Scenario 17 targets the qualified DENY (`UNLESS routed through apply_transition()`): the task explicitly bypasses the exception, making the DENY unconditional; both approaches handle this correctly at 100%. Scenario 18 requires semantic PII recognition: the rule says "logging of PII fields" without enumerating field names; the model must infer that `originator_name` and `account_number` are PII.